

The Grafana Dashboard

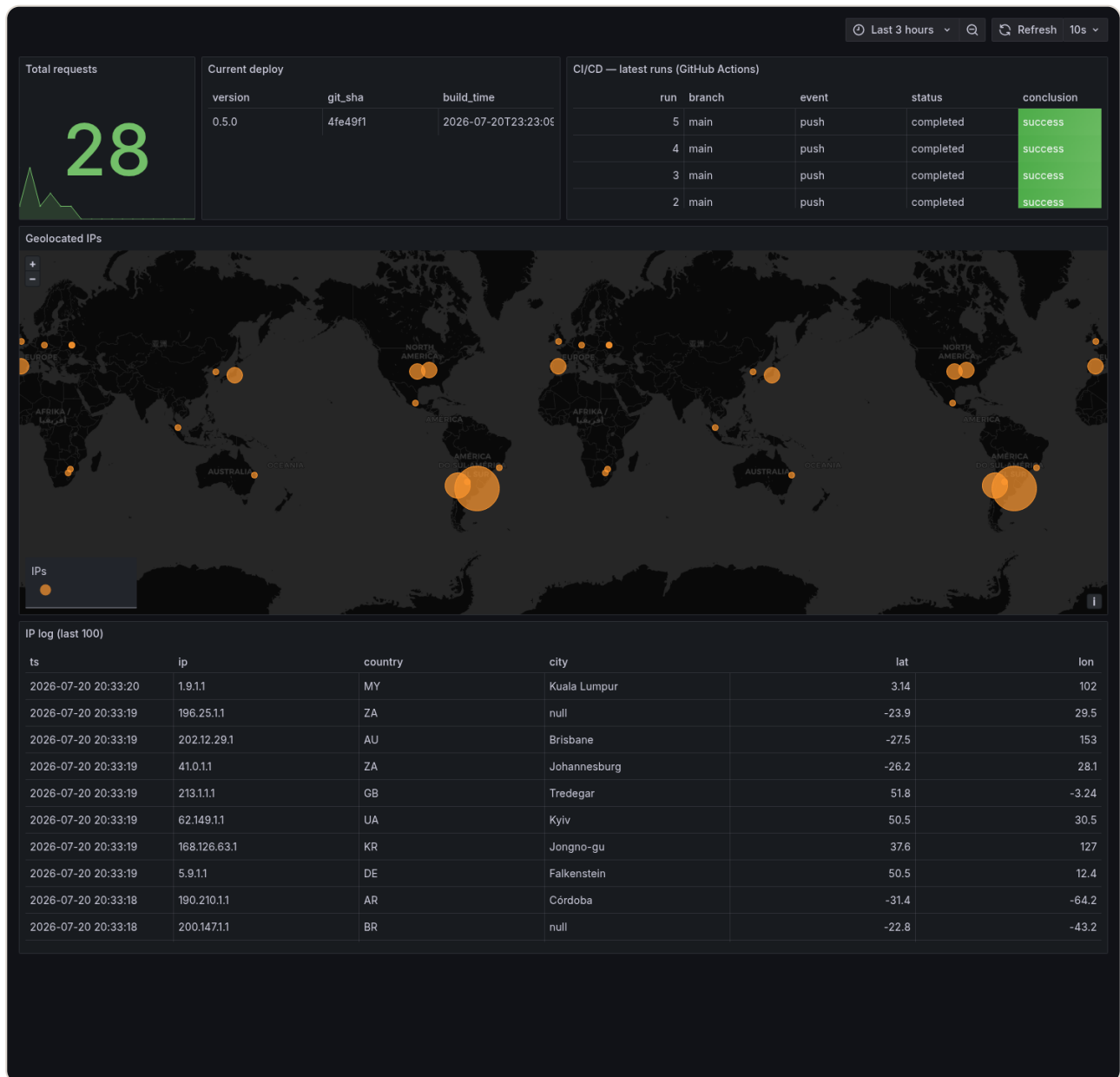
A live, public map of geolocated IPs and CI/CD status - provisioned entirely
as code

Carlos Pereyra
July 20, 2026

The dashboard is the most visual part of the lab. It shows, in real time, where the requests to the app come from on a world map, the last IPs seen, the currently deployed version, and the status of the latest CI/CD runs. It is dedicated to the lab, exposed publicly with anonymous read access, and defined completely as code - datasource, dashboard and provisioning all live in the repository.

FIELD	VALUE
URL	devops-lab-grafana.pereyra.ar
Grafana	dedicated instance (deploy/grafana.yaml), NodePort 30898
Datasource	Infinity (yesoreyeram-infinity-datasource)
Data source of truth	the app's own JSON endpoints + the GitHub API
Access	anonymous Viewer (public)
Definition	grafana/ (datasource, dashboard JSON, provider)

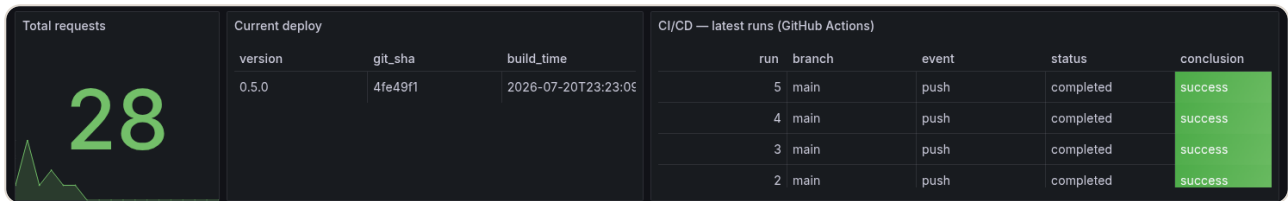
Live dashboard



1. Panels

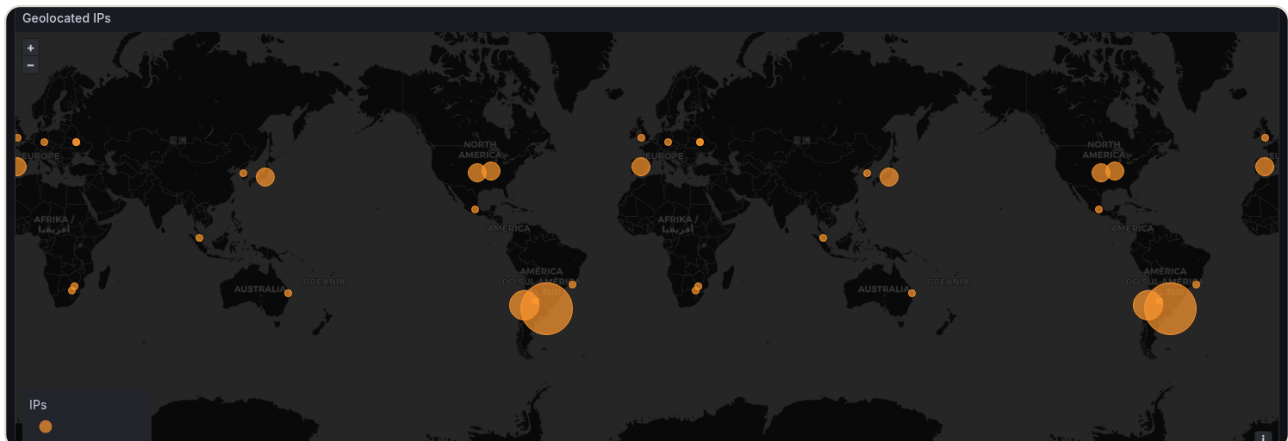
The dashboard is a single screen with five panels: three status tiles at the top, the map, and the log table.

Top row - status at a glance



- Total requests - a stat panel summing the count across all map points.
- Current deploy - the running version, commit SHA and build time, read from the app's /version endpoint (baked into the image at build time). This tells you exactly what is live.
- CI/CD - latest runs - the last GitHub Actions runs (number, branch, event, status, conclusion), read from the app's /ci endpoint (which proxies and caches the GitHub API). The conclusion cell is colour-mapped: green for success, red for failure. The dashboard literally shows the health of its own pipeline.

Geolocated IPs - the map



A Geomap panel plots one marker per location, using coordinate mode (latitude/longitude from the app), with the marker size scaled by how many requests hit that spot. Hovering a marker shows a tooltip with country, city, count and the IP(s) seen there - so any point on the map can be traced back to a concrete address. The data comes from the app's /stats endpoint, which aggregates seen requests by rounded coordinates.

IP log

IP log (last 100)					
ts	ip	country	city	lat	lon
2026-07-20 20:33:20	1.9.1.1	MY	Kuala Lumpur	3.14	102
2026-07-20 20:33:19	196.25.1.1	ZA	null	-23.9	29.5
2026-07-20 20:33:19	202.12.29.1	AU	Brisbane	-27.5	153
2026-07-20 20:33:19	41.0.1.1	ZA	Johannesburg	-26.2	28.1
2026-07-20 20:33:19	213.1.1.1	GB	Tredeggar	51.8	-3.24
2026-07-20 20:33:19	62.149.1.1	UA	Kyiv	50.5	30.5
2026-07-20 20:33:19	168.126.63.1	KR	Jongno-gu	37.6	127
2026-07-20 20:33:19	5.9.1.1	DE	Falkenstein	50.5	12.4
2026-07-20 20:33:18	190.210.1.1	AR	Córdoba	-31.4	-64.2
2026-07-20 20:33:18	200.147.1.1	BR	null	-22.8	-43.2

A table of the last 100 requests (timestamp, IP, country, city, latitude, longitude), read from the app's /log endpoint. It refreshes every 10 seconds, like the rest of the dashboard.

2. How it is fed (Infinity datasource)

There is no Prometheus and no database behind this dashboard - the application is its own data source. The Infinity datasource fetches JSON directly from HTTP endpoints and turns it into table data that any panel can render. Three sources feed the panels: the app's /stats and /log endpoints, and the GitHub Actions API.

The datasource (provisioned)

```
apiVersion: 1
datasources:
- name: ipecho-json
  uid: ipecho-json
  type: yesoreyeram-infinity-datasource
  access: proxy
  isDefault: true
```

How a panel queries the app

The map panel points Infinity at the app's /stats endpoint and maps JSON fields to columns:

```
{
  "type": "json", "source": "url", "format": "table",
  "url": "http://ipecho/stats",
  "columns": [
    { "selector": "lat",      "type": "number" },
    { "selector": "lon",      "type": "number" },
    { "selector": "count",    "type": "number" },
    { "selector": "country",  "type": "string" },
    { "selector": "city",     "type": "string" },
    { "selector": "ips",      "type": "string" }
  ]
}
```

The CI/CD panel uses the same datasource against the app's http://ipecho/ci endpoint. That endpoint proxies the GitHub Actions API and caches the result for a few minutes - the dashboard refreshes every 10 seconds, so querying GitHub directly would blow through its unauthenticated rate limit (60 requests/hour); the cache keeps it to a handful per hour. Because ipecho and Grafana run in the same namespace, the app is reachable at the short DNS name http://ipecho.

3. Provisioned as code

Nothing about this dashboard is clicked together by hand. The Grafana instance is a plain Deployment; the datasource, dashboard provider and dashboard JSON are mounted from ConfigMaps built from the repository. Re-applying the manifests recreates the exact same dashboard - it is reproducible and reviewable.

```
grafana/  
  datasources/infinity.yaml      # the Infinity datasource  
  dashboards-provider/provider.yaml # tells Grafana where to load dashboards  
  dashboards/ipecho.json        # the dashboard itself  
deploy/grafana.yaml             # the Grafana Deployment + Service
```

One command builds the ConfigMaps from those files, creates the admin Secret and deploys:

```
make grafana-deploy
```

The result: a public, self-contained dashboard that turns the app's own JSON into a live world map, a request log, the running version, and its own CI/CD status - with no extra monitoring stack, and fully reproducible from the repository.